Assignment 3: LevelDB

Make changes in a large code base

COP290 - Design Practices

Group Size: This assignment must be done in a **group of two**.
Deadline: TBA.

Contents

1 Background **2**
- 1.1 LevelDB Overview . . . 2
- 1.2 LSM-tree Overview . . . 2

2 Project Description **3**
- 2.1 Range Scan . . . 3
- 2.2 Range Delete . . . 4
- 2.3 Manual Full Compaction . . . 4
- 2.4 Deliverables . . . 5

3 Environment Setup **6**
- 3.1 Getting the Source Code . . . 6
- 3.2 Building LevelDB . . . 6
- 3.3 Basic Usage of LevelDB APIs . . . 6
- 3.4 Cleaning the Database Directory . . . 7

4 Evaluation Rubric **8**
- 4.1 Evaluation Criteria . . . 8
- 4.2 Evaluation Method . . . 8

5 Submission Instructions **9**

1 Background

1.1 LevelDB Overview

LevelDB is a high-performance, embedded **key-value store** developed by Google. A key-value store manages data as pairs of ($key \rightarrow value$), where each key uniquely identifies a value. The system does not enforce any additional schema or relational structure.

Applications interact with LevelDB through a minimal interface consisting of three primary operations:

- `Put(key, value)`: Inserts a new key-value pair or updates the value of an existing key.
- `Get(key)`: Retrieves the most recent value associated with the given key.
- `Delete(key)`: Removes the value associated with the given key.

The external API is intentionally simple. However, internally LevelDB implements a storage engine based on the **Log-Structured Merge (LSM) tree** design.

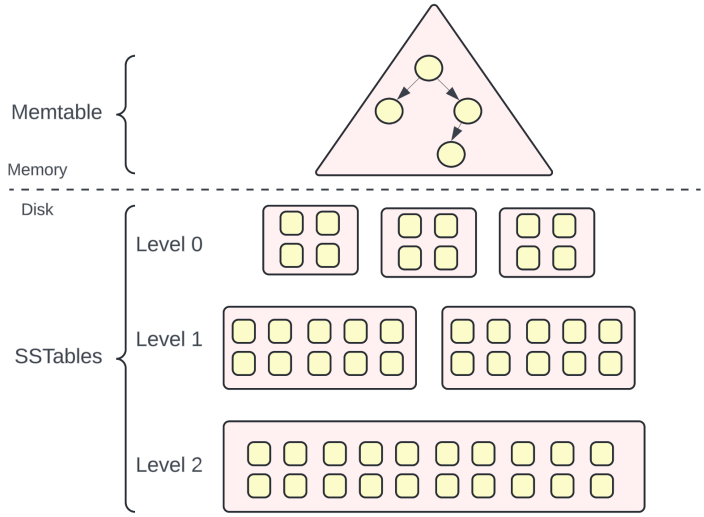

Figure 1: High-level structure of an LSM-tree. Data is first written to the MemTable in memory, flushed to Level 0 SSTables, and progressively merged into lower levels through compaction.

1.2 LSM-tree Overview

Traditional storage engines update data *in place*. These updates often involve random disk writes, which can significantly degrade write performance. In contrast, LevelDB is based on the **LSM-tree**, a storage design that improves write performance by buffering updates in memory and converting small random writes into large sequential writes.

Figure 1 illustrates high-level structure of an LSM-tree. The LSM-tree consists of an in-memory structure called the **MemTable** and multiple levels of on-disk files known as **SSTables** (Sorted String Tables). Each level in the LSM-tree has a size limit, and the

size limit increases exponentially for deeper levels. When the amount of SSTables in a level exceeds its size limit, a background process called **compaction** is triggered.

When a write operation is issued, updates are first inserted into the MemTable in main memory. The MemTable temporarily buffers updates in memory and maintains keys in sorted order. Once the MemTable exceeds a predefined threshold, it is marked immutable and flushed to disk as a new SSTable in **Level 0**. Because these SSTables are created independently from MemTable flushes, SSTables in Level 0 may contain overlapping key ranges.

As more SSTables accumulate in Level 0, the system triggers a **compaction**. During compaction, one or more SSTables from Level 0 are selected and merged with SSTables from the next level. The key-value pairs from these SSTables are sorted and merged to produce new SSTables in **Level 1**. Unlike Level 0, SSTables in deeper levels of the LSM-tree (Level 1 and below) are organized such that their key ranges do not overlap and together partition the key space of the level. This process maintains the structural properties of the LSM-tree while gradually moving data to deeper levels.

2 Project Description

The goal of this project is to understand the internal design of LevelDB by analyzing its write path and compaction mechanism, and by implementing modifications to the LevelDB storage engine.

Students will study how write operations propagate through the LSM-tree-based storage engine and how compaction is triggered and executed. Based on this understanding, students will implement two modifications to the LevelDB codebase.

2.1 Range Scan

Many applications require retrieving multiple key-value pairs within a specified key interval. For example, a system may need to retrieve all records whose keys fall between two boundaries. LevelDB primarily supports point lookups through the following API:

```
Status DB::Get(const ReadOptions&, const Slice&, std::string*)
```

The `Get` operation retrieves the value associated with a single key by searching the MemTable and SSTables across multiple levels of the LSM-tree. However, applications often require retrieving multiple keys within a range. In this assignment, students must implement the following API:

```
Status DB::Scan(const ReadOptions&,
                const Slice& start_key,
                const Slice& end_key,
std::vector<std::pair<std::string, std::string>>* result)
```

The function should return all key-value pairs whose keys fall within the following half-open interval:

```
[start_key, end_key)
```

Students are expected to implement this functionality using the **iterator interface** provided by LevelDB. The iterator interface allows sequential traversal of key-value pairs stored in the database. The scan should begin at `start_key` and stop once `end_key` is reached. For each key encountered during the scan, the returned value must correspond to the **latest visible version** of that key, consistent with the semantics of the `Get` operation. If no keys exist within the specified range, the function should return an empty result vector.

2.2 Range Delete

LevelDB provides an API for deleting individual keys:

```
Status DB::Delete(const WriteOptions&, const Slice& key)
```

This operation removes a single key from the database. However, many applications require deleting a contiguous range of keys with a single operation. In this assignment, students must implement the following API:

```
Status DB::DeleteRange(const WriteOptions&,
                       const Slice& start_key,
                        const Slice& end_key)
```

This function should logically delete all key value pairs whose keys fall within the following half-open interval:

```
[start_key, end_key)
```

Since SSTables in LevelDB are immutable, keys cannot be physically removed from existing files once they are written. Therefore, when a `Delete` operation is issued, the key is not immediately removed from existing SSTables. Instead, a deletion marker is recorded, and the corresponding key-value pair is physically removed only when it is processed during **compaction**.

Likewise, range deletions should follow similar semantics. Keys that fall within an active deletion range must be discarded when SSTables are merged during compaction and must not appear in the resulting SSTables.

2.3 Manual Full Compaction

LevelDB automatically performs compaction in the background based on its internal scheduling policy. However, in many storage systems it is also useful to allow applications to manually trigger compaction operations. Manual compaction can be used to reorganize the database, remove obsolete data, or force data to be merged across levels. In this part of the project, students will implement a **manual full compaction mechanism** in LevelDB. Students must add a new API of the following form:

```
Status DB::ForceFullCompaction()
```

When this function is invoked, the database should perform a full compaction synchronously. The function should return only after the full compaction has completed. During this operation, foreground operations, including reads and writes, may be blocked until the compaction finishes.

When a full compaction is triggered, the system should iterate through all levels of the LSM-tree and trigger compaction sequentially starting from Level 0 toward deeper levels. This ensures that recently flushed data is progressively merged into lower levels and that obsolete entries can be removed during the compaction process. Students may reuse or extend the existing compaction infrastructure in LevelDB to implement this functionality.

In addition, students must collect and report basic **compaction statistics**. These statistics should summarize the work performed during the compaction process. The following information should be reported:

- Number of compactions executed
- Number of input files
- Number of output files
- Total bytes read during compaction
- Total bytes written during compaction

The collected statistics must be printed in a human-readable format when the manual compaction operation completes.

2.4 Deliverables

Students must submit the following items:

- Modified LevelDB source code implementing:
  - Range Scan
  - Range Delete
  - Manual Full Compaction
- A report (`report.pdf`) describing:
  - Analysis of the LevelDB write path and compaction mechanism
  - Design and implementation of the `RangeScan(start_key, end_key)` API
  - Design and implementation of the `DeleteRange(start_key, end_key)` API
  - Design and implementation of the manual full compaction mechanism
  - Evaluation of the implemented features, including compaction statistics

All explanations and analyses must be grounded in the LevelDB source code. Students must clearly reference the relevant files and functions used in their analysis. Reports that contain only high-level descriptions without code references will not receive credit.

3 Environment Setup

This section provides basic instructions for obtaining the LevelDB source code, building the project, and running the benchmarking tool used in this assignment.

3.1 Getting the Source Code

First, clone the official LevelDB repository from GitHub:

```
git clone --recursive https://github.com/google/leveldb.git
cd leveldb
```

3.2 Building LevelDB

LevelDB uses CMake as its build system. The following commands build the project in release mode:

```
cmake -DCMAKE_BUILD_TYPE=Release -DCMAKE_CXX_STANDARD=17 ..
cmake --build . -j
```

3.3 Basic Usage of LevelDB APIs

LevelDB is typically used as an embedded library inside a C++ program. A database instance must first be opened, after which the application can invoke APIs such as `Put`, `Get`, and `Delete`.

A simple example is shown below:

```
#include <iostream>
#include <string>
#include "leveldb/db.h"

int main() {
    leveldb::DB* db;
    leveldb::Options options;
    options.create_if_missing = true;

    leveldb::Status status = leveldb::DB::Open(options, "/tmp/testdb", &db);
    if (!status.ok()) {
        std::cerr << "Failed to open database: " << status.ToString() <<
            std::endl;
        return 1;
    }

    leveldb::WriteOptions write_options;
    leveldb::ReadOptions read_options;

    // Put
    db->Put(write_options, "key1", "value1");
    db->Put(write_options, "key2", "value2");
```

```
    // Get
    std::string value;
    status = db->Get(read_options, "key1", &value);
    if (status.ok()) {
        std::cout << "key1 => " << value << std::endl;
    }

    // Delete
    db->Delete(write_options, "key2");

    delete db;
    return 0;
}
```

The same usage pattern applies to the APIs implemented in this assignment. Once added to the LevelDB interface, the new APIs can be invoked through the same database object in a similar way.

3.4 Cleaning the Database Directory

LevelDB stores its data in a directory specified when opening the database. If the directory already contains a database created in a previous run, the existing data will be reused.

When testing your implementation, it is helpful to remove the existing database directory and start with a fresh database.

For example:

```
rm -rf /tmp/testdb
```

4 Evaluation Rubric

4.1 Evaluation Criteria

Your submission will be evaluated based on the following criteria:

Component	**Weightage**	**Description**
Write, Read Path & Compaction Analysis	15%	Clear explanation of the LevelDB write path, read path, and compaction mechanism based on the source code, including how `Put`, `Get`, and iterator-based operations access data across MemTables and SSTables and how compaction reorganizes data across levels.
Range Scan	25%	Correct implementation of the `Scan(start_key, end_key)` API.
Range Delete	25%	Correct implementation of the `DeleteRange(start_key, end_key)` API.
Manual Full Compaction	25%	Correct Implementation of manual full compaction and reporting of compaction-related statistics from the storage engine.
Report Quality	10%	Clarity, organization, and proper referencing of relevant source code in the report.

4.2 Evaluation Method

The correctness of your implementation will be evaluated using a set of TA-designed test cases. These tests will interact with the database through the APIs defined in this assignment under various scenarios.

For testing purposes, students must **strictly follow the API specifications provided in the assignment description**. Any deviation from the required API signatures may result in the implementation failing the evaluation tests. Students should ensure that their implementation correctly integrates with the existing LevelDB codebase and works with the standard `Put`, `Get`, and `Delete` APIs.

5 Submission Instructions

1. You need to submit your modified LevelDB source code, and `report.pdf`.
2. The project must compile using the original LevelDB build procedure. **DO NOT** modify the build system or introduce external libraries or dependencies.
3. Before packaging your submission, remove all compiled binaries and build artifacts (e.g., run `make clean` or delete the `build` directory). Submissions containing unnecessary build files or binaries may receive a penalty.
4. Do not include database directories (e.g., `/tmp/leveldbtest`) or other generated files in your submission archive.
5. We will use modern AI-based plagiarism checkers. If there is a match, you will receive **zero** in this assignment and another assignment of our choosing.
6. There may additionally be a **demo and viva**. It is possible that the assignment works correctly but the viva performance falls short of expectations. In this case, marks may be significantly reduced.
7. The report should describe your design decisions and include your GitHub repository link. **Both team members must commit code from their own GitHub accounts — no exceptions.**
8. All submissions will be via **Moodle**. Create a tar file as follows:
   (a) Assume all your files are in a directory named `project`.
   (b) Go to the parent directory of the project.
   (c) Run: `tar -cvf <entry_number>.tar project`
   (d) An entry number is of the form: `2025ANZ8223` (It is **NOT** your user id or email id).
   (e) Submit this tar file on Moodle.
9. Failure to follow the required submission format may result in a penalty.
10. Submit several hours before the deadline. Double-check your submission.
11. **No emails will be entertained.**